数据分析与算法实践课程 · 实验报告（选题二）

基于 MovieLens 的残差协同过滤电影推荐系统

先看大众口碑，再找个人口味：从基础评分到残差协同过滤

一、提交人与贡献说明

角色	姓名	学号	贡献说明
组长	温洛	42512131	完成实验设计、双偏差与残差 UserCF/ItemCF 建模、时序验证、TUI 开发和报告整理。

二、实验内容与工作概览

本次选择实验内容二，用公开电影评分数据做一个协同过滤推荐系统。数据来自 MovieLens ml-latest-small，包含 100,004 条评分、671 位用户和 9,066 部有评分电影。我们主要做了三件事：

- **把数据整理好，再寻找口味相近的人和电影：**用连续位置映射替代原始 `movieId` 下标，以 NaN 表示缺失评分并用布尔掩码隔离；先拟合正则化用户/电影双偏差基线 $b_{ui} = \mu + b_u + b_i$，再在残差上计算带共同评分门槛与收缩的余弦相似度，融合 UserCF 与 ItemCF 两条修正通道。
- **按时间检验推荐效果：**按每位用户的时间戳做外层 80%/20% 留出（训练 79,748 条，测试 20,256 条），并在训练集内部再留出验证集（16,209 条）调参。测试集 RMSE 从用户均分基线的 0.9750 降至双偏差的 0.9072，最终混合模型为 0.8979；MAE 从 0.7477 降至 0.6851。另报告随机邻居阴性对照、合成信号阳性对照、逐项消融和按用户配对 bootstrap 区间。
- **做成能实际操作的终端程序：**TUI 基于 prompt_toolkit，支持用户、流派子串和片名子串筛选，F2/F3 翻页，Tab 切换输入框，Ctrl-Q 退出，直接读取离线 Top-50 缓存。

1　先认识数据，再处理实现细节

1.1　MovieLens 数据集结构与评分分布

同样给一部电影打 5 分，有人是因为这部片公认不错，有人则是格外喜欢它的风格。我们的目标，就是尽量把大众口碑和个人口味拆开来看。实验使用 GroupLens 发布的 MovieLens ml-latest-small 数据集[1]：100,004 条带时间戳的显式评分，671 位用户（每人至少 20 条评分），9,066 部至少被评分一次的电影，电影目录共 9,125 部。每部电影由 `movieId` 唯一标识，`movies.csv` 提供英文片名、上映年份和竖线分隔的流派标签。数据中有 19 个命名流派（Action、Adventure、Animation、Children、Comedy、Crime、Documentary、Drama、Fantasy、Film-Noir、Horror、IMAX、Musical、Mystery、Romance、Sci-Fi、Thriller、War、Western），另有 18 部影片标记为 `(no genres listed)`。

评分采用 0.5 分至 5.0 分的半星十档制。全站平均分 $\mu = 3.5436$，标准差约 1.0581。不低于 3.0 分的正向评价共 82,170 条，占 82.17%；4.0 分是众数档位（28,750 条，占 28.75%），其次为 3.0 分（20,064 条，占 20.06%）和 5.0 分（15,095 条，占 15.09%）。2.0 分及以下共 13,385 条（13.38%），最低档 0.5 分有 1,101 条（1.10%）。评分主要集中在中高分段。用户通常会挑自己感兴趣的电影来看，因此这些分数本来就带有选择偏好。建模时，我们先估计电影和用户各自的评分倾向，再寻找更细的个人偏好。

1.2　直接建评分矩阵，会碰到四个问题

最开始很容易想到：把用户放在行上，把 `movieId` 放在列上，直接做评分矩阵。但这个写法有几个实际问题，我们逐一处理：

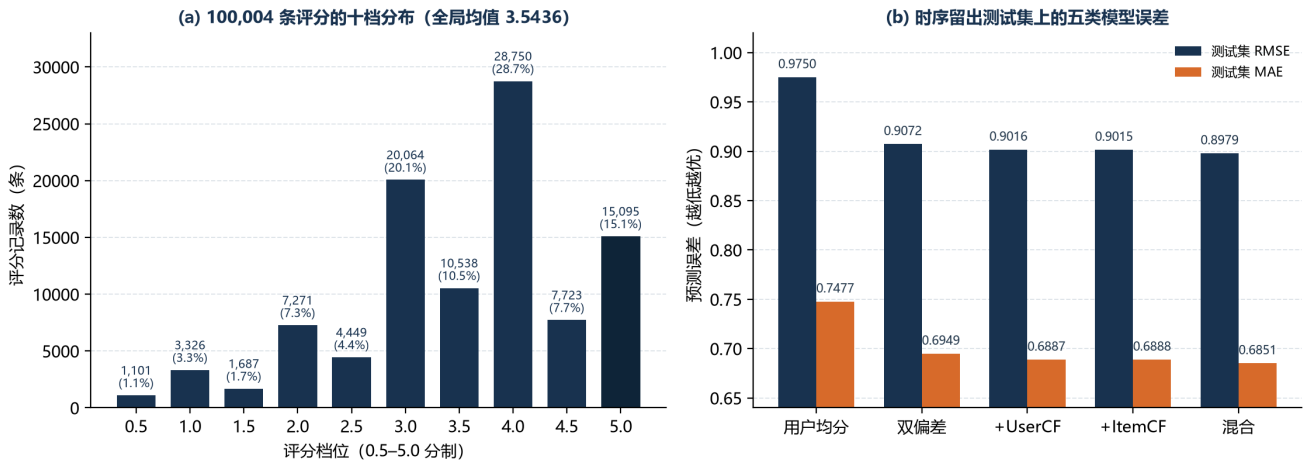


Figure 1: （a）MovieLens ml-latest-small 100,004 条评分的十档频数分布；（b）按用户时间顺序留出测试集（20,256 条）上五类模型的 RMSE 与 MAE。

1. **电影 ID 不连续，矩阵会变得很大。**有评分的电影只有 9,066 部，但评分表中的最大 `movieId` 为 163,949。若直接构造 $(671, 163949)$ 的双精度矩阵，单个矩阵约 880.08 MB；其中 154,883 列（94.47%）从未出现评分。改为连续位置映射后，671×9066 的 float32 矩阵约 24.33 MB。
2. **没看过和打低分是两回事。**若把评分缩放到 $[0, 1]$ 并把未评分位置填 0，0.5 分会与“缺失”得到相同的数值，相似度和加权预测会把未评分当成真实低分。实现用 `NaN` 表示缺失，并额外保存布尔观测掩码；残差矩阵只在矩阵乘法前把缺失位置置零，观测掩码负责隔离这些位置。
3. **取推荐结果时，要先认清行列。**矩阵行轴对应用户、列轴对应电影。若写成 `user_prediction[:, userid]`，取到的是一部电影对所有用户的预测分，再把用户行号当作片名下标，推荐结果与目标用户无关。正确做法是沿用户行取 `pred[u_idx, :]`，过滤已评分电影后再用索引反查片名与流派。
4. **训练和测试得用同一套索引与模型。**若训练集和测试集各自建矩阵、各自训练，再跨矩阵比较预测值与测试评分，模型并没有在“只看过去”的条件下预测未来，指标不可解释。实现采用按用户时间戳的嵌套切分：外层训练/测试，内层训练/验证；所有超参数只在内层验证集选择。

2 算法：先估计基础分，再寻找个人口味

2.1 为什么只比较评分还不够

UserCF 的思路很简单：找出评分习惯相近的用户，再参考他们对某部电影的看法。设用户 u 的历史均分为 $\bar{r}_u$，用户 u 与 v 的相似度为 s_{uv}，经典中心化预测写成：

$$\hat{r}_{ui}^{(\text{legacy})} = \bar{r}_u + \frac{\sum_{v \in N(u)} s_{uv}(r_{vi} - \bar{r}_v)}{\sum_{v \in N(u)} |s_{uv}|}.$$

这种做法已经考虑了“有人打分严、有人打分松”，但还会受到电影口碑的影响。比如两个人都喜欢一部公认的佳作，不一定代表他们口味相近。于是我们换个比较方式：先扣掉电影和用户的常见评分倾向，再看两个人在哪些电影上有相似的意外惊喜或失望。

2.2 第一阶段：正则化双偏差基线

第一步是给每条评分算一个“正常情况下大概会打多少分”的基础值。它由全局平均分、用户打分的松紧和电影整体口碑组成：

$$b_{ui} = \mu + b_u + b_i.$$

其中 μ 是训练集评分均值；$b_u > 0$ 说明这位用户打分相对宽松，$b_i > 0$ 说明这部电影普遍得分较高。评分少的用户和冷门电影容易出现不稳定的估计，因此加入 L_2 正则化参数 λ_u, λ_i，交替更新 10 轮：

$$b_u \leftarrow \frac{\sum_{i \in I(u)} (r_{ui} - \mu - b_i)}{|I(u)| + \lambda_u}, \qquad b_i \leftarrow \frac{\sum_{u \in U(i)} (r_{ui} - \mu - b_u)}{|U(i)| + \lambda_i}.$$

验证集最终选出 $\lambda_u = \lambda_i = 5.0$。直观地说，一个人平时就爱打高分，一部电影也普遍受欢迎，这两件事先算进基础分；接下来才轮到“这位用户是否特别喜欢它”。

2.3 第二阶段：残差、收缩相似度与阻尼修正

从训练集真实评分中扣除基础分，得到个体偏好残差：

$$e_{ui} = r_{ui} - b_{ui} = r_{ui} - (\mu + b_u + b_i), \quad (u,i) \in \Omega_{\text{train}}.$$

比如一个平时打分严格的人，给一部评价普通的冷门片打了高分，这个“比预期更喜欢”的部分就是正残差。两个人若经常在相同电影上出现同方向的残差，可能比单看原始评分更有共同口味。我们只比较两人都看过的电影，即交集 $I_{uv} = I(u) \cap I(v)$，计算残差余弦相似度，并加入共同评分门槛 $c_{\min}$ 和收缩系数 β_s：

$$s_{uv} = \mathbf{1}(|I_{uv}| \geq c_{\min}) \cdot \frac{|I_{uv}|}{|I_{uv}| + \beta_s} \cdot \frac{\sum_{i \in I_{uv}} e_{ui} e_{vi}}{\sqrt{\sum_{i \in I_{uv}} e_{ui}^2} \sqrt{\sum_{i \in I_{uv}} e_{vi}^2}},$$

其中分母为 0 时相似度取 0。实现中先用 $C = MM^{\mathsf{T}}$ 统计共同评分数，再用零填充残差矩阵的 $E_{\text{dot}} = R_{\text{res}} R_{\text{res}}^{\mathsf{T}}$ 计算内积；缺失位置的残差为 0，不会贡献内积，交集由重叠计数和掩码保证。随后只保留正相似度，按行取 Top-K 邻居。

对目标电影 i，在给该片评分且与目标用户正相关的候选中取 Top-K 邻居集合 $N_K(u)$，用验证集选择的阻尼系数 α_u 控制近邻信号强度：

$$\Delta_u(u,i) = \frac{\sum_{v \in N_K(u) \cap U(i)} s_{uv} e_{vi}}{\sum_{v \in N_K(u) \cap U(i)} s_{uv}}, \qquad \hat{r}_{ui}^{(\text{UserCF})} = \text{clip}(b_{ui} + \alpha_u \Delta_u(u,i),\, 0.5,\, 5.0)\,.$$

举个例子：基础分为 3.80，邻居给出的平均残差信号是 $+0.70$。如果全盘接受，就会预测 4.50；乘上阻尼系数 $\alpha_u = 0.2$ 后，只加 0.14，预测为 3.94。邻居的意见有用，但不必照单全收。

2.4 ItemCF 与双通道混合

沿电影方向对称计算物品间收缩残差相似度 $s_{ij}^{(\text{item})}$。为避免 $9{,}066 \times 9{,}066$ 的稠密相似度矩阵，实现先用余弦最近邻为每部电影取 128 个候选，再只对候选对统计共同评分人数、做收缩并保留 Top-K。目标电影 i 的物品残差修正为：

$$\Delta_i(u,i) = \frac{\sum_{j \in N_K(i)} s_{ij}^{(\text{item})} e_{uj}}{\sum_{j \in N_K(i)} s_{ij}^{(\text{item})}}.$$

双通道融合后：

$$\hat{r}_{ui}^{(\text{Hybrid})} = \text{clip}(b_{ui} + \gamma\,(\alpha_u \Delta_u(u,i) + \alpha_i \Delta_i(u,i))\,, 0.5, 5.0)\,.$$

我们分别调整 UserCF、ItemCF 的邻居数、共同评分门槛和阻尼强度，再用验证集选择混合系数 γ。最后使用的参数见表 1。

Table 1: 验证集选择的模型参数

组件	参数	取值
双偏差	用户/电影正则化	$\lambda_u = \lambda_i = 5.0$（交替 10 轮）
UserCF	K / $c_{\min}$ / β_s / α_u	320 / 3 / 15.0 / 0.2
ItemCF	K / $c_{\min}$ / β_s / α_i	100 / 1 / 15.0 / 0.25
混合	混合系数 γ	0.75

3 评估协议与对照实验

3.1 时序切分与调参

推荐系统最需要回答的问题是：面对用户之后会给出的评分，模型究竟猜得怎样？我们按时间切分数据，用较早的评分训练，留出较晚的评分测试。Ferrari Dacrema 等人在 RecSys 2019 的研究也提醒我们，复杂模型未必胜过调好参数的简单方法[2]。因此这里把用户均分、双偏差与残差协同放在同一套测试条件下比较。

评估按每位用户的 `timestamp`（同时间按 `movieId`）升序排序，取前 80% 作外层训练集（79,748 条），最近 20% 作测试集（20,256 条）；再对外层训练集做同样的 80%/20% 切分，得到内层训练集与验证集（16,209 条）。双偏差、UserCF、ItemCF 和混合系数都只在内层验证集上选择，外层测试集只在最终评估时使用。调参网

格覆盖双偏差正则化、UserCF/ItemCF 的邻居数、共同评分门槛、收缩系数与阻尼强度，以及混合系数；具体网格和选择结果见 `evaluation_results.json`。

效果用测试集 RMSE 和 MAE 衡量，数值越低越好。我们还按用户进行 2,000 次配对 bootstrap，给出“基线误差减去模型误差”的 95% 区间；区间整体大于 0 时，说明在这套重采样条件下，提升方向比较稳定。

3.2 主结果与对照/消融

表 2 汇总测试集结果。空白对照使用全局均值；阴性对照在保持每个用户邻居权重分布与度的前提下随机置换邻居身份，重复 25 次；阳性对照使用一个 3×3 的合成残差矩阵，基线分 3.0，注入同向邻居信号后预测 5.0，增益 2.0，判定阈值为“高于基线 1.0 以上”。

Table 2: 测试集指标与对照/消融结果（20,256 条留出评分）

实验类型	模型配置	RMSE	MAE	说明
空白对照	全局均值 μ	1.0647	0.8423	不含任何个性化项。
基线	用户历史均分 $\bar{r}_u$	0.9750	0.7477	只消除用户打分松紧差异。
基线	双偏差 $\mu + b_u + b_i$	0.9072	0.6949	同时剥离用户与电影主效应。
加法	双偏差 + UserCF	0.9016	0.6887	用户残差通道，$\alpha_u = 0.2$。
互补	双偏差 + ItemCF	0.9015	0.6888	物品残差通道，$\alpha_i = 0.25$。
加法	双偏差 + UserCF + ItemCF	0.8979	0.6851	双通道混合，$\gamma = 0.75$。
减法	混合模型去掉 UserCF	0.9018	0.6895	仅保留 ItemCF 修正。
减法	混合模型去掉 ItemCF	0.9024	0.6898	仅保留 UserCF 修正。
减法	混合模型去掉两项	0.9072	0.6949	退回双偏差基线。
阴性对照	随机邻居（25 次均值）	0.9091	0.6958	随机化邻居身份，保留权重分布与度。
阳性对照	合成 3×3 残差	3.0→5.0	—	增益 2.0，通过（阈值 1.0）。

结果可以分三步看。首先，双偏差相对用户均分基线把 RMSE 从 0.9750 降至 0.9072，降幅 6.96%；MAE 从 0.7477 降至 0.6949，降幅 7.07%。最大的一步提升来自双偏差基础分。接着，在双偏差上单独加入 UserCF 或 ItemCF，RMSE 分别降至 0.9016（相对双偏差降 0.62%）和 0.9015（降 0.63%）；两条通道同时加入后为 0.8979，相对双偏差降 1.03%，相对用户均分基线累计降 7.91%，MAE 累计降 8.37%。最后，我们逐项关掉协同通道：从混合模型中去掉 UserCF 或 ItemCF，RMSE 分别回升到 0.9018 和 0.9024；两项都去掉则回到 0.9072。

配对 bootstrap 的 95% 区间均不含 0：UserCF 相对双偏差的 ΔRMSE 为 $[0.0037, 0.0076]$，ItemCF 为 $[0.0035, 0.0080]$，混合模型为 $[0.0067, 0.0122]$。阴性对照的 25 次随机邻居平均 RMSE 为 0.9091（5%–95% 分位 0.9082–0.9101），比双偏差的 0.9072 更差，也比真实 UserCF 的 0.9016 差 0.0075 左右；把邻居身份打乱后，预测反而变差；真正有用的是邻居之间的评分关系，而不只是权重数值。阳性对照从 3.0 提升到 5.0，说明在预先构造的同向信号场景中，残差检索可以识别对应的邻居。

值得留意的是，混合模型比双偏差又降低了 0.0093 RMSE，幅度不大，却能在配对 bootstrap 和对照实验中看到一致的方向。这里的 0.8979 是测试集上的评分预测误差；至于推荐列表的排序是否更符合喜好，还要用 Recall@K、NDCG@K 等指标进一步检查，见第 5 节。

4 工程实现与交互界面

4.1 模块划分与向量化实现

项目由三个 Python 模块组成：`recommender.py` 负责数据读取、时序切分、双偏差/相似度/残差修正、调参、对照实验和缓存构建；`tui.py` 负责全屏终端界面与单用户文本输出；`main.py` 提供 `--once`、`--evaluate`、`--rebuild`、`--user` 等命令行入口。启动演示时直接读取 `data/ml-latest-small/demo_data.json`，不需要联网或重新训练。

底层以 NumPy 向量化为主：评分矩阵为 float32 的 $671 \times 9{,}066$ 数组，缺失位置是 NaN；M 为布尔观测掩码。用户共同评分数通过 $C = MM^{\mathsf{T}}$ 得到，残差内积通过 $E_{\text{dot}} = R_{\text{res}} R_{\text{res}}^{\mathsf{T}}$ 得到；ItemCF 先用 scikit-learn 的余弦最近邻为每部电影取 128 个候选，再只对候选对统计共同评分人数，最后用 CSR 稀疏矩阵保存 Top-K 有向物品邻居。这样就不用保存 $9{,}066^2$ 的完整物品相似度矩阵，缺失评分也不会被算成 0 分。

内存方面，原始 ID 直接建列的 float64 矩阵约 880.08 MB；实现的 float32 密集评分矩阵约 24.33 MB，残差 CSR 约 0.80 MB（100,004 个非零元），布尔观测掩码约 0.50 MB。缓存 `demo_data.json` 保存数据集计数、选中参数、测试指标以及每位用户的 Top-50 未看电影推荐。演示缓存用全部 100,004 条评分重建，方便现场浏览推荐结果；第 3 节的测试指标则来自只用外层训练集（79,748 条）训练的模型。两套结果用途不同：一套用于演示，一套用于评价效果。

4.2 TUI 布局与交互

为了不让结果只留在一张指标表里，我们做了一个 TUI。切换用户、筛选流派、搜索片名都可以直接操作，还能看到每部电影的预测分和邻居支持权重。界面使用 prompt_toolkit 实现全屏应用，依赖已写入 `requirements.txt`。界面包含四个区域：

1. 顶部信息区：显示评分总数、用户数、有评分电影数，以及当前用户的评分条数、平均分和高分电影摘要。
2. 检索控制栏：三个文本框分别接收用户 ID（Enter 校验并切换）、流派子串（大小写不敏感）和片名子串（大小写不敏感）。
3. 推荐表格与分页器：每页 8 条，列为排名、片名、流派、预测分和支持权重；从该用户的 Top-50 缓存中筛选，已评分电影不会出现。
4. 底部状态与快捷键栏：显示筛选结果数、当前页范围和 F2 上页、F3 下页、Tab/Shift-Tab 切换输入框、Ctrl-Q/Ctrl-C 退出；界面启用鼠标支持。

底部还会显示 UserCF 邻居数、ItemCF 邻居数、混合系数，以及混合模型在测试集上的 RMSE/MAE。表格中的“支持权重”指 UserCF 与 ItemCF 邻居权重之和；它和残差对预测分的实际贡献是两个数值，均在构建缓存时提前计算。



Figure 2: TUI 运行界面：用户 66 的缓存推荐（第 1 页，每页 8 条）。

4.3 用户 66 的推荐案例

用户 66 在数据中有 49 条评分，平均分 3.959。缓存中的前四名及分解如表 3 所示；表中 Δ 为已经乘过混合系数 $\gamma = 0.75$ 的残差贡献，因此“基础分 + 两列贡献”等于预测分（四舍五入误差除外）。《马耳他之鹰》的基础分 4.482 低于《肖申克的救赎》，ItemCF 贡献 +0.201 使其升至 4.692；《肖申克的救赎》基础分 4.632 最高，但 ItemCF 贡献为负（-0.052），退居第四。可以看到，电影本来的口碑和用户的独特偏好会一起影响排名。这是用户 66 的一个具体例子，整体效果还可以对照第 3 节的测试结果来看。

Table 3: 用户 66 的前四条缓存推荐及分量分解

排名	电影	预测分	基础分	UserCF Δ	ItemCF Δ
1	Maltese Falcon, The (1941)	4.692	4.482	+0.008	+0.201
2	Godfather: Part II, The (1974)	4.640	4.551	+0.020	+0.069
3	It Happened One Night (1934)	4.625	4.483	+0.044	+0.099
4	Shawshank Redemption, The (1994)	4.592	4.632	+0.012	-0.052

5 结论、局限与改进方向

这次实验从整理评分数据开始，先用双偏差抓住用户与电影的一般规律，再用残差 UserCF/ItemCF 补上个人口味，最后用时序测试和 TUI 看结果。原始 ID 直接建列的 float64 矩阵约 880.08 MB，改为 float32 连续索引后约 24.33 MB；测试集 RMSE 从用户均分基线的 0.9750 降至双偏差的 0.9072，最终混合模型为 0.8979，MAE 从 0.7477 降至 0.6851。配对 bootstrap、随机邻居对照和合成信号实验也给出了相互呼应的结果：残差协同确实带来一些额外帮助，只是提升幅度比较小。

还有几件事值得接着做。目前我们主要看 RMSE/MAE，尚未测推荐列表的 Top-K 排序效果；时间切分也只有一轮。ItemCF 先筛 128 个候选邻居，可能错过部分长尾电影；时间衰减、内容特征和冷启动问题也还没纳入模型。下一步可以补上 Recall@K、NDCG@K、覆盖率与多轮时间切分，再加入时间衰减，并和 ItemKNN 等简单基线继续比较。

参考文献与数据来源

1. F. Maxwell Harper and Joseph A. Konstan. The MovieLens Datasets: History and Context. ACM Transactions on Interactive Intelligent Systems, 5(4), 2015. Dataset: https://grouplens.org/datasets/movielens/.
2. Maurizio Ferrari Dacrema, Paolo Cremonesi, and Dietmar Jannach. Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches. In Proceedings of ACM RecSys' 19, 2019. https://arxiv.org/abs/1907.06902.